

SenaContigo 2.

Sí. Tomando el documento como **fuentes funcional principal**, yo plantearía SENAContigo como una aplicación institucional de seguimiento longitudinal, no simplemente como un sistema de encuestas.

El documento define una idea arquitectónica muy importante: **variables dinámicas** → **encuestas** → **respuestas históricas** → **necesidades** → **casos** → **seguimiento** → **estadísticas**, y explícitamente propone combinar *Form Builder, Survey Engine, Dynamic Variables, Longitudinal Responses, Case Management, Rules Engine y Analytics*.

A partir de eso, este sería un planteamiento de nivel **Senior / Arquitectura de Software**.

1. Visión arquitectónica de SENAContigo

Objetivo

Construir una plataforma web que permita al SENA:

1. Identificar aprendices afectados por una situación de emergencia.
2. Registrar su situación inicial.
3. Volver a consultar periódicamente su evolución.
4. Incorporar nuevas variables sin modificar el software.
5. Mantener el historial completo de respuestas.
6. Detectar necesidades automáticamente.
7. Crear y gestionar casos de atención.
8. Asignar responsables.
9. Registrar acciones y seguimientos.
10. Obtener indicadores históricos por aprendiz, ficha, programa, centro, regional y SENA.
11. Medir la evolución y recuperación de los aprendices.

El documento establece que **no se debe sobrescribir el estado anterior**; por ejemplo, una vivienda puede pasar de *Afectada* a *Inhabitable* y posteriormente a *En reparación*, conservando todas las mediciones.

2. Principios arquitectónicos

Yo establecería estos principios desde el comienzo:

P1. Historial inmutable

Una respuesta registrada no debe modificarse para representar una nueva medición.

18/08 → Vivienda = Afectada

25/08 → Vivienda = Inhabitable

10/09 → Vivienda = En reparación

Cada registro representa una medición diferente.

P2. Configuración sobre código

Las nuevas variables, opciones y encuestas deben poder crearse desde el sistema administrativo.

Por ejemplo:

Crear variable:

COMPUTADOR

Crear opciones:

Tiene computador

Tiene computador pero presenta daños

Comparte computador

No tiene computador

Sin desplegar una nueva versión del aplicativo.

P3. No eliminar información histórica

Las opciones antiguas deben desactivarse, no eliminarse.

activa = false

El documento plantea precisamente este mecanismo para preservar la interpretación histórica.

P4. Separación de responsabilidades

Separar:

Situación observada

↓

Necesidad

↓

Caso

↓

Acciones

↓

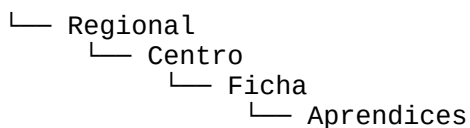
Seguimiento

Una vivienda inhabitable es un **dato**, mientras que requerir alojamiento temporal es una **necesidad**.

P5. Multi-tenant lógico institucional

La seguridad debe respetar:

SENA



Los permisos dependen del nivel organizacional del usuario.

3. Requerimientos funcionales nivel Senior

RF-001 — Gestión de Regionales

El sistema deberá permitir administrar:

- Código regional.
- Nombre.
- Estado.
- Centros asociados.

El documento establece que cada regional tiene un CODIGO_REGIONAL único.

RF-002 — Gestión de Centros de Formación

Cada centro deberá manejar:

- CODIGO_CENTRO
- Nombre.
- Regional asociada.
- Estado.

Relación:

REGIONAL 1 ——— N CENTROS

4. Gestión académica

RF-003 — Programas de formación

Registrar:

- Código del programa.

- Versión.
- Nombre.
- Nivel de formación.
- Estado.

Los niveles definidos en el documento son:

- Auxiliar
 - Curso especial
 - Operario
 - Técnico
 - Tecnólogo
-

RF-004 — Gestión de fichas

Una ficha deberá manejar:

FICHA_CHARACTERIZACION
FECHA_INICIAL
FECHA_FINAL
ESTADO_FICHA
CODIGO_CENTRO
CODIGO_PROGRAMA

Estados:

- Cancelada
 - En ejecución
 - Terminada
 - Terminada por fecha
 - Terminada por unificación
-

5. Gestión de aprendices

RF-005 — Registro de aprendiz

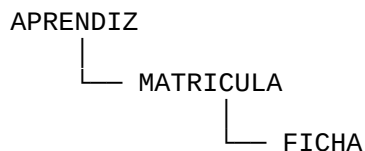
Información mínima:

id_unico
tipo_documento
numero_documento

nombres
apellidos
celular
correo

RF-006 — Matrícula del aprendiz

Debe existir una entidad intermedia:



Con estados:

- Matriculado
 - En formación
 - Aplazado
 - Condicionado
 - Cancelado
 - Retirado
 - Desertado
 - Certificado
 - No certificado
-

6. Gestión de usuarios y seguridad

RF-007 — Roles

Los roles definidos son:

SuperAdmin

Acceso completo a todos los centros.

Dirección

Acceso a todos los centros de su regional.

Coordinador

Acceso únicamente a su centro.

Instructor

Acceso únicamente a los grupos a los que está asignado.

Aprendiz

Acceso a su información y posibilidad de registrar nuevas mediciones, pero no modificar históricos.

7. Motor de variables dinámicas

Este es, arquitectónicamente, uno de los módulos más importantes.

RF-008 — Categorías

Ejemplos iniciales:

Código	Categoría
VIVIENDA	Vivienda
TRANSPORTE	Transporte
CONECTIVIDAD	Tecnología
FAMILIA	Familia
EMPLEO	Laboral
ECONOMIA	Económica
ALIMENTACION	Alimentación

RF-009 — Variables

Una variable debe tener como mínimo:

```
id
categoria_id
nombre
codigo
descripcion
tipo_respuesta
obligatoria
activa
fecha_creacion
```

Pero como arquitecto Senior yo agregaría:

```
id
categoria_id
codigo
```

nombre
descripcion
tipo_respuesta
version_actual
es_sensible
es_obligatoria
activa
created_at
updated_at
created_by
updated_by

8. Versionamiento de variables

RF-010 — Versionar variables

No basta con `activa`.

Debe existir:

VARIABLE

- VERSION 1
- VERSION 2
- VERSION 3

Esto permite que una misma variable cambie su definición sin romper los datos históricos.

El documento recomienda explícitamente manejar versiones de variables para preservar qué pregunta y opciones existían en cada momento.

9. Opciones de variables

RF-011 — Gestión de opciones

Una opción deberá manejar:

id
variable_version_id
codigo
texto
valor_numerico
orden
nivel_afectacion
activa

Ejemplo:

VIVIENDA
NORMAL → 0
AFECTADA → 1

INHABITABLE → 2
DESTRUIDA → 3

10. Escalas de afectación

RF-012 — Nivel de afectación configurable

Inicialmente:

0 = Sin afectación
1 = Leve
2 = Moderada
3 = Grave
4 = Crítica

Pero **no lo programaría directamente en código**.

Debe ser configurable.

11. Constructor de encuestas

RF-013 — Crear encuesta

Permitir:

Nombre
Descripción
Tipo
Fecha inicio
Fecha final
Estado

El documento plantea explícitamente el concepto de encuesta como una medición temporal.

RF-014 — Constructor de preguntas

Una encuesta podrá seleccionar variables:

ENCUESTA

- Vivienda
- Transporte
- Conectividad
- Empleo

Y otra:

ENCUESTA

- Alimentación
- Salud

12. Segmentación

RF-015 — Crear segmentos dinámicos

Este requerimiento es fundamental.

El sistema debe permitir seleccionar:

Todos los aprendices
o:

Regional
Centro
Ficha
Programa
Aprendices afectados

El documento incluso plantea consultas como:

volver a preguntar únicamente a aprendices con vivienda afectada.

Yo lo convertiría en un **motor de segmentación reutilizable**.

13. Respuestas longitudinales

RF-016 — Registrar respuesta

Una respuesta debe asociar:

aprendiz
variable
variable_version
opcion
encuesta
fecha
usuario

Idealmente también:

source
created_at
ip
device

si las políticas institucionales lo permiten.

14. Prohibición de sobrescritura

RF-017 — Protección histórica

El sistema deberá impedir que:

respuesta 18/08

sea reemplazada por:

respuesta 25/08

La nueva consulta siempre genera un nuevo registro.

15. Estado actual vs. histórico

RF-018 — Obtener estado actual

El sistema debe poder calcular:

Estado actual

tomando la última medición válida.

Pero también:

Histórico completo

y:

Evolución

El documento diferencia precisamente entre estado actual y evolución temporal.

16. Motor de indicadores

RF-019 — Estadísticas

Indicadores mínimos:

Por aprendiz

Situación actual

Evolución

Índice de afectación

Casos abiertos

Necesidades

Por ficha

Total aprendices

Afectados

No afectados
Casos críticos
Evolución

Por centro

Por regional

SENA nacional

17. Cortes históricos

RF-020 — Cortes de encuesta

Esta decisión debe formar parte de la arquitectura desde el inicio.

Las estadísticas deben poder calcularse para:

18/08
25/08
10/09
25/09

permitiendo comparar:

ANTES → DESPUÉS

El documento recomienda explícitamente que las estadísticas se basen en **cortes de encuesta** y no únicamente en el último valor.

18. Índice de afectación

RF-021 — Motor de puntuación

Ejemplo:

Vivienda	3
Transporte	2
Conectividad	1
Familia	2
Empleo	3
Economía	3
Alimentación	1

Total 15

Pero los pesos deben ser configurables.

Vivienda	= 5
Empleo	= 3
Conectividad	= 1

19. Motor de necesidades

RF-022 — Identificación de necesidades

Separar:

VARIABLE

de:

NECESIDAD

Ejemplo:

Vivienda = Inhabitable

↓

Necesidad = Alojamiento temporal

20. Motor de reglas

RF-023 — Reglas configurables

Ejemplo:

SI

VIVIENDA = INHABITABLE

ENTONCES

Crear necesidad:

ALOJAMIENTO

Prioridad:

CRÍTICA

Crear caso:

SITUACIÓN HABITACIONAL

Asignar:

BIENESTAR

Otro:

SI

CONECTIVIDAD = SIN INTERNET

AND

PROGRAMA = ADSO

ENTONCES

Crear alerta:

RIESGO CONTINUIDAD FORMATIVA

21. Gestión de casos

RF-024 — Crear caso

Un caso debería contener:

id
aprendiz_id
tipo
prioridad
estado
fecha_creacion
fecha_cierre
responsable_id
origen

Estados recomendados:

NUEVO
ASIGNADO
EN_ATENCION
EN_ESPERA
ESCALADO
RESUELTO
CERRADO
CANCELADO

22. Necesidades

RF-025 — Catálogo de necesidades

Ejemplos:

Alojamiento
Alimentación
Transporte
Conectividad
Apoyo económico
Atención familiar
Continuidad formativa

El documento establece explícitamente necesidades, casos, acciones y seguimientos como entidades centrales.

23. Acciones

RF-026 — Gestión de acciones

Cada caso podrá tener:

Acción
Responsable
Fecha compromiso
Fecha ejecución
Estado
Observaciones
Evidencia

24. Seguimiento

RF-027 — Seguimiento del caso

Ejemplo:

Caso: Alojamiento temporal
18/08 → Caso creado
19/08 → Contactado
20/08 → Solicitud enviada
21/08 → Alojamiento asignado
25/08 → Seguimiento

25. Auditoría

RF-028 — Auditoría completa

Registrar:

quién
qué hizo
cuándo
sobre qué entidad
valor anterior
valor nuevo
IP

Especialmente para:

- usuarios
- permisos
- variables
- versiones
- encuestas
- respuestas
- reglas
- casos.

26. Requerimientos no funcionales

Aquí es donde un proyecto comienza a diferenciarse de una aplicación CRUD convencional.

RNF-001 Seguridad

Implementar:

- RBAC.
 - Control de acceso por Regional/Centro/Ficha.
 - JWT/OAuth2/OIDC.
 - Password hashing.
 - Protección contra OWASP Top 10.
 - Rate limiting.
 - Auditoría.
 - Gestión segura de sesiones.
-

RNF-002 Protección de datos

El sistema manejará información personal de aprendices.

Por tanto:

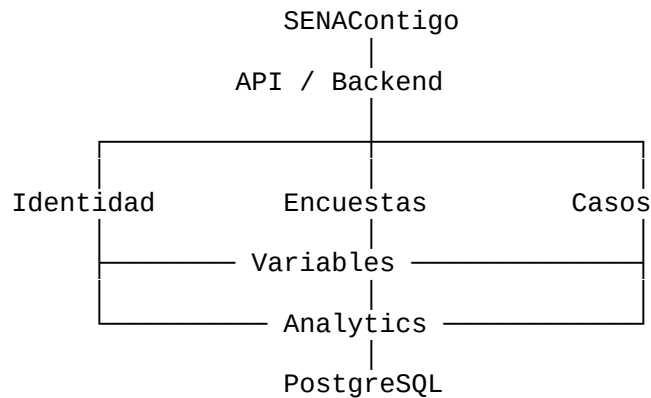
- mínimo privilegio;
 - cifrado en tránsito;
 - cifrado de información sensible cuando corresponda;
 - trazabilidad;
 - políticas de retención;
 - control de exportaciones;
 - anonimización para analítica cuando sea necesaria.
-

27. Arquitectura propuesta

No empezaría con microservicios.

Para este proyecto recomendaría inicialmente:

Modular Monolith



Con módulos independientes internamente.

Módulos

identity
organization
academic
apprentices
variables
surveys
responses
segments
rules
needs
cases
actions
followups
analytics
audit
notifications

Esto permite mantener una arquitectura profesional sin introducir desde el inicio la complejidad operacional de microservicios.

28. Stack tecnológico recomendado

Teniendo en cuenta el tipo de aplicación, propondría:

Backend

Python
FastAPI
SQLAlchemy 2.x
Pydantic 2.x
PostgreSQL
Redis
Celery / ARQ

Frontend

React
Vite
TypeScript
React Router
TanStack Query
o mantener Vue 3 si el equipo ya está estandarizado en Vue.

Infraestructura

Docker
Nginx / Caddy
PostgreSQL
Redis
CI/CD
GitHub/GitLab

29. Modelo de datos inicial

El documento propone como mínimo:

categorias
variables
variable_versiones
opciones_variable
encuestas
encuesta_variables
respuestas
aprendices
fichas
programas
sedes
casos
necesidades
acciones
seguimientos
reglas

Yo lo ampliaría a:

regionales
centros
programas
fichas
aprendices
matriculas
usuarios
roles
permisos
usuario_roles
categorias
variables
variable_versiones
opciones_variable
encuestas
encuesta_variables

segmentos
respuestas
reglas
regla_condiciones
regla_acciones
necesidades
casos
caso_necesidades
acciones
seguimientos
indicadores
pesos_indicadores
cortes_encuesta
notificaciones
auditoria

30. Plan profesional de desarrollo

Yo lo dividiría en **10 fases**.

Fase 0 — Arquitectura y descubrimiento

Duración: 1 semana

Entregables:

- visión del sistema;
 - alcance;
 - arquitectura;
 - modelo de dominio;
 - mapa de actores;
 - matriz de permisos;
 - modelo inicial de datos;
 - ADRs;
 - backlog inicial.
-

Fase 1 — Fundación técnica

1 semana

Construir:

Repositorio

Docker
CI/CD
Backend
Frontend
PostgreSQL
Redis
Configuraciones
Logging
Manejo de errores
Entregable:

Aplicación desplegable con pipeline CI/CD.

Fase 2 — Organización SENA

1–2 semanas

Implementar:

Regional
Centro
Programa
Ficha
Aprendiz
Matrícula

Y:

SuperAdmin
Dirección
Coordinador
Instructor
Aprendiz

Fase 3 — Motor de variables

2 semanas

Construir:

Categorías
Variables
Versiones
Opciones
Niveles de afectación

Aquí debe aparecer el primer gran **MVP técnico**.

Fase 4 — Survey Engine

2 semanas

Implementar:

Encuestas
Preguntas
Variables
Segmentación
Publicación
Respuestas

Y especialmente:

Respuesta histórica

Fase 5 — Dashboard y Analytics

2 semanas

Dashboard:

SENA
Regional
Centro
Ficha
Aprendiz

Indicadores:

Afectados
No afectados
Nivel de afectación
Evolución
Índice

Fase 6 — Case Management

2 semanas

Construir:

Necesidades
Casos
Responsables
Acciones
Seguimientos
Estados
Prioridades

Fase 7 — Rules Engine

2 semanas

Construir un motor configurable:

```
IF
  condición
THEN
  acción
```

Por ejemplo:

```
VIVIENDA = INHABITABLE
      ↓
NECESIDAD = ALOJAMIENTO
      ↓
CASO = CRÍTICO
      ↓
RESPONSABLE = BIENESTAR
```

Fase 8 — Notificaciones

1 semana

Canales posibles:

Email
SMS
WhatsApp
Notificaciones internas
Pero los canales deben ser abstraídos:

```
NotificationService
├── EmailProvider
├── SMSProvider
└── WhatsAppProvider
```

Fase 9 — Seguridad, auditoría y pruebas

2 semanas

Pruebas:

Unitarias
Integración
API
RBAC
Seguridad
Carga
Regresión
E2E

Además:

Audit Trail
Backup
Restore
Monitoring
Alertas

Fase 10 — Piloto

2–3 semanas

No desplegaría inicialmente para todo el SENA.

Haría:

1 Regional

↓

2 Centros

↓

Varias fichas

↓

Grupo controlado de aprendices

Mediría:

- rendimiento;
- usabilidad;
- errores;
- calidad de datos;
- tiempos de respuesta;
- adopción;
- calidad de indicadores.

Después:

Piloto

↓

Correcciones

↓

Versión 1.0

↓

Expansión

31. Backlog inicial

Yo convertiría el proyecto en épicas:

Épica	Prioridad
Arquitectura	P0
Seguridad	P0
Regionales/Centros	P0
Programas/Fichas	P0
Aprendices	P0
Roles y permisos	P0
Variables dinámicas	P0
Versionamiento	P0
Encuestas	P0
Respuestas históricas	P0
Dashboard	P1
Segmentación	P1
Necesidades	P1
Casos	P1
Seguimientos	P1
Rules Engine	P1
Índice de afectación	P1
Notificaciones	P2
Integraciones externas	P2
IA	P3

32. Qué NO desarrollaría inicialmente

Esto es importante.

No empezaría por:

- Inteligencia artificial.
- Chatbot.
- WhatsApp.
- App móvil nativa.
- Microservicios.
- Machine Learning.
- GIS avanzado.
- Predicción.

Primero construiría correctamente:

Datos SENA

↓
Variables
↓
Encuestas
↓
Histórico
↓
Necesidades
↓
Casos
↓
Seguimiento
↓
Indicadores

La IA puede incorporarse posteriormente sobre una base de datos histórica bien estructurada.

33. MVP real de SENAContigo

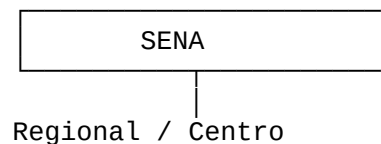
Para mí el **MVP profesional** debería contener solamente:

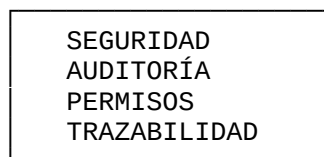
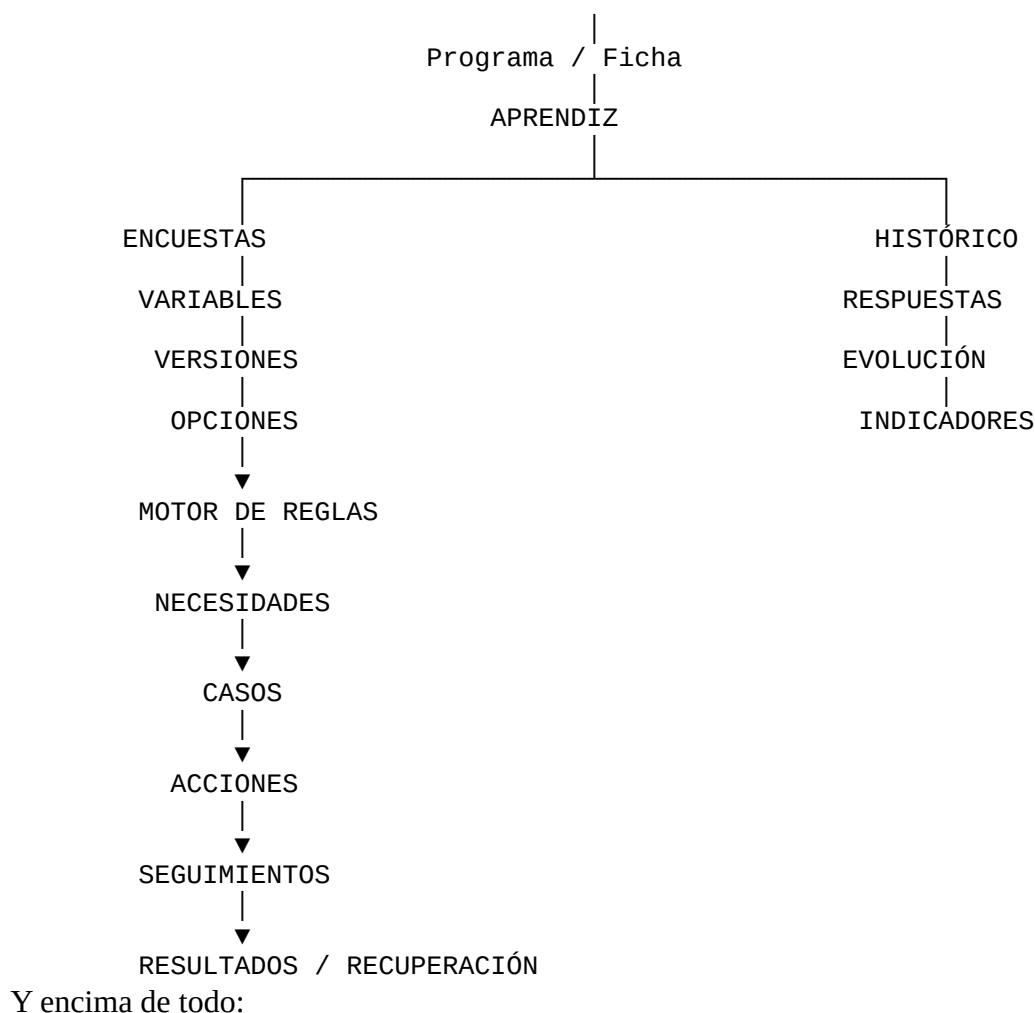
1. Usuarios
2. Roles
3. Regionales
4. Centros
5. Programas
6. Fichas
7. Aprendices
8. Matrículas
9. Categorías
10. Variables
11. Versiones
12. Opciones
13. Encuestas
14. Segmentación
15. Respuestas históricas
16. Dashboard
17. Casos
18. Necesidades
19. Seguimientos
20. Auditoría

Con eso ya tendríamos un producto funcional y arquitectónicamente sólido.

34. La arquitectura conceptual final

La representaría así:





Mi recomendación como siguiente paso

No empezaría todavía a programar. El siguiente entregable debería ser un **Documento de Arquitectura de Software (SAD) de SENAContigo**, mucho más preciso que el documento funcional que tienes actualmente.

Ese documento debería definir, en orden:

1. **Arquitectura C4** — Context, Container, Component y Deployment.
2. **Casos de uso completos.**
3. **Matriz de roles × permisos × alcance territorial.**
4. **Modelo entidad-relación completo.**
5. **Modelo de datos PostgreSQL**, incluyendo PK, FK, índices, constraints y auditoría.

6. **Contratos REST de la API.**
7. **Estructura del proyecto FastAPI + React/TypeScript.**
8. **Flujos de encuesta y respuestas históricas.**
9. **Diseño del Rules Engine.**
10. **Diseño del Case Management.**
11. **Arquitectura de Analytics y cortes históricos.**
12. **Requisitos de seguridad.**
13. **Estrategia de pruebas.**
14. **Docker + CI/CD + ambientes DEV/QA/PROD.**
15. **Backlog técnico en épicas → historias de usuario → criterios de aceptación.**
16. **Roadmap de implementación por sprint.**

La base conceptual del documento que adjuntaste es suficientemente buena para dar ese siguiente salto: incluso ya identifica las entidades centrales categorías, variables, variable_versiones, opciones_variable, encuestas, encuesta_variables, respuestas, aprendices, fichas, programas, sedes, casos, necesidades, acciones, seguimientos y reglas.

Ese SAD sería el documento que usaría un equipo Senior para comenzar realmente el desarrollo de SENAContigo, evitando que el proyecto termine convertido en un CRUD de encuestas difícil de evolucionar.